

Mini-Spec: a ancoragem de 5 minutos antes do vibe coding

Pesquisa sobre a mini-especificação de uma página que ancora o Claude Code antes do primeiro prompt, motivada pela prática "O Especialista em Claude Code e o Builder que 'Manda o Agente Codar Direto'": por que vibe coding sem contexto vira retrabalho estrutural e como 5 minutos de spec se transformam em economia de tokens.

Fernando Melo · 21 de setembro de 2026

01 · O PROBLEMA

Um prompt curto e "manda ver" — até a terceira funcionalidade

Lucas usa o Claude Code no esquema vibe coding: escreve um prompt curto, tipo "cria um app de agendamento de consultas", e deixa o agente se virar. Pra ele, parar pra escrever um PRD (Product Requirement Document) de 10 páginas antes de codar é burocracia que só atrasa quem quer entregar rápido — e ele questiona o que ganha de verdade escrevendo uma spec antes.

A resposta dada na prática: no início de um projeto ou num MVP super simples, um PRD de 10 páginas realmente parece burocracia inútil. Mas, à medida que o app cresce, a "vibe" cobra o seu preço. O ganho real de escrever uma especificação — mesmo uma mini-spec de uma página — não é burocracia, é controle de contexto e economia de tokens.

02 · O CUSTO DO VIBE CODING PURO

Três dores que uma spec de uma página evita

Sem spec: o preço da "vibe"

Ping-pong de tokens: prompt vago → o Claude assume, você corrige, ele reescreve o código anterior — ciclo de tentativa e erro que consome milhares de tokens.

"Espaguete de IA": sem arquitetura pensada, na 3ª ou 4ª função nova (pagamento, cancelamento, notificações) a IA quebra o que ela mesma fez antes.

Regras de negócio ocultas: o Claude sabe criar a tela, mas não sabe se dois pacientes podem agendar o mesmo horário ou se há prazo de cancelamento — e inventa uma regra padrão.

Com mini-spec: o retorno de 5 minutos

Acerta de primeira: uma especificação clara reduz drasticamente o custo e o tempo de processamento.

Arquitetura com norte: a spec ajuda a desenhar banco de dados e APIs prevendo o futuro, evitando o "castelo de cartas".

Regras explícitas: definir antes evita consertar lógica de negócio espalhada pelo código depois.

Vibe coding puro vs. spec-driven

ASPECTO	VIBE CODING PURO	SPEC-DRIVEN (COM MINI-SPEC)
Velocidade inicial	Instantânea — o código sai em segundos	Exige de 10 a 20 minutos de reflexão escrita
Aproveitamento de tokens	Baixo — gasta muito corrigindo rumos	Alto — o agente vai direto ao ponto
Escalabilidade do app	Baixa — tende a quebrar após poucas telas	Alta — a arquitetura nasce sólida
Resultado final	Um MVP genérico (o que a IA acha que é certo)	Um software sob medida para o seu problema

Mini-Spec Ultra-Direta: um Markdown de bullet points

Para não cair na burocracia, a especificação precisa ser focada em dados e regras, não em formatação. A estrutura recomendada para o Claude Code é um arquivo Markdown simples (spec.md), com tópicos — o Claude "adora" listas estruturadas. Ela começa pelo fluxo principal e pelas restrições que a IA não tem como adivinhar.

```
# Spec: Sistema de Agendamento (Fase 1)

### 1. Objetivo Principal
Criar um MVP onde [Tipo de Usuário, ex: Pacientes] possam
agendar [ex: consultas médicas] com [ex: Médicos].

### 2. Regras de Negócio Ocultas (o que a IA não adivinha)
* Concorrência: dois usuários não podem agendar o mesmo horário
  com o mesmo profissional.
* Janela de Tempo: agendamentos só com no mínimo 24h de antecedência.
* Duração: cada consulta tem duração fixa de 30 minutos.
* Horário Comercial: seg. a sex., das 08h às 18h.

### 3. Modelo de Dados Mínimo (Entidades)
* User/Patient: id, nome, email, telefone.
* Professional: id, nome, especialidade.
* Slot (Horários): id, professional_id, data_hora_inicio, status.
* Appointment: id, patient_id, slot_id, criado_em.

### 4. Telas / Fluxo de Cliques (Escopo)
* Tela 1: lista de profissionais disponíveis.
* Tela 2: calendário só com horários 'disponível' do profissional.
* Tela 3: confirmação com resumo do agendamento.
```

- 1 Crie um arquivo na raiz do projeto (ex.: agendamento.md).
- 2 Gaste ~3 minutos preenchendo os pontos acima com a sua ideia.
- 3 No terminal do Claude Code, rode um prompt curto apontando para o arquivo.

"Claude, leia o arquivo agendamento.md e crie a estrutura inicial do app seguindo exatamente essas regras e o modelo de dados."

Reduzir o espaço de busca da IA

O Claude e outros agentes não são humanos criativos que precisam de 50 páginas de inspiração — são motores de previsão estatística. Um prompt vago faz o modelo adivinhar com base em tutoriais genéricos que já viu publicamente; entregar o Markdown ultra-direto reduz o espaço de busca da IA.

- 1 **Ancoragem de Contexto:** antes de escrever qualquer linha de código, o Claude consulta as restrições (ex.: "janela de 24h"), impedindo decisões aleatórias na lógica de backend.
- 2 **Geração One-Shot:** em vez de criar o banco no prompt 1, a API no prompt 3 e a validação no prompt 5, a arquitetura inteira nasce de uma vez, já sabendo como as tabelas vão se relacionar.
- 3 **Menos alucinação:** a IA inventa quando faltam dados; dizendo exatamente os campos que o banco precisa ter, ela não precisa inventar regra da cabeça.

O Claude vai seguir 100% perfeito de primeira? Raramente. Mas os erros mudam de categoria: sem spec, o erro é estrutural (o banco não suporta o que você quer, e é preciso refatorar o app inteiro); com mini-spec, o erro é superficial (falta um botão, um alinhamento CSS) — corrigível com um prompt de 5 palavras. Um retrabalho de 1 hora vira um ajuste de 30 segundos.

A mini-spec customizada: agendamento médico

Para deixar o teste prático pronto, bastaram três coordenadas: quem agenda e quem atende (Pacientes e Médicos), como funciona o horário (fixo, 30 minutos, com o médico definindo a própria grade) e onde acontece o atendimento (presencial com endereço, ou online por link de vídeo). Com isso, saiu a spec abaixo, já pronta para colar no terminal.

```
# Spec: Sistema de Agendamento Médico (Fase 1)

### 1. Objetivo Principal
Pacientes marcam consultas com Médicos. Suporta atendimento
presencial (endereço físico) e online (link de vídeo).

### 2. Regras de Negócio e Validações
* Duração Fixa: toda consulta tem exatamente 30 minutos.
* Gestão de Grade: o médico define sua própria grade de horários.
* Modalidade: a consulta deve ser "Presencial" ou "Online".
* Concorrência: dois pacientes não podem agendar o mesmo slot
  com o mesmo médico.
* Antecedência Mínima: agendar/cancelar com no mínimo 24h.

### 3. Modelo de Dados Mínimo (Entidades)
* User (Paciente): id, nome, email, telefone.
* Doctor (Médico): id, nome, especialidade, CRM, endereco_consultorio.
* TimeSlot (Grade): id, doctor_id, data_hora_inicio/fim, status.
* Appointment: id, patient_id, doctor_id, timeslot_id,
  modalidade (ENUM: presencial|online), link_video (opcional),
  criado_em.

### 4. Telas e Fluxo do App
* Tela 1 (Busca): médicos por especialidade, com endereço se atender
  presencial.
* Tela 2 (Calendário): TimeSlots 'disponível'; escolhe horário e
  modalidade.
* Tela 3 (Confirmação): resumo; link de vídeo se Online, endereço
  se Presencial.
```

"Claude, leia o arquivo spec-agendamento.md. Crie a estrutura inicial deste app (banco de dados, rotas de backend e telas descritas no fluxo) seguindo rigorosamente todas as regras de negócio e validações listadas."

Três motivos que separam sucesso de loop infinito de refatoração

- A Acerta o alvo:** sem requisitos explícitos, o Claude calcula a probabilidade do que um "app genérico" deveria ter; com a spec, calcula a probabilidade do que o seu app precisa ter — o desperdício de energia vai a zero.
- B Evita o scope creep:** agentes são proativos demais; sem fronteira definida, o Claude pode criar tabelas de histórico médico, avaliação por estrelas ou prontuário eletrônico — coisas que ninguém pediu e que só quebram o código principal.
- C Sobrevive a sessões longas:** janelas de contexto limpam mensagens antigas ("context drift"); uma diretriz dada só no primeiro prompt é esquecida após 20-30 interações. Persistir a spec em arquivo (spec.md ou CLAUDE.md) garante que a IA releia o objetivo a cada nova tarefa.

O conceito de PRD burocrático se transforma em ferramenta de ancoragem de código: um mapa que o Claude pode tropeçar em detalhes de CSS, mas nunca perder o rumo do destino combinado.